

Parte 1 – Visão Geral do Projeto

Objetivo da API

O objetivo principal da API é conectar os dados do back-end e enviar para o banco, sendo necessário configurar toda a rota, o banco escolhido foi o MongoDB, foi utilizado o Mongoose para configurar as rotas do código, sobre as rotas foi utilizado:

- GET(ALL) para obter todas as informações
- GET(One) para obter apenas um dado específico
- POST para enviar os dados
- PUT para atualizar algum dado
- DELETE para apagar algum dado

Problema que ela resolve

O projeto, trata-se de um sistema de gestão de livros, onde permite o usuário cadastrar livros, autores e a sua categoria, o usuário consegue obter as informações vinda do banco de dados, assim como atualizar ou apagar algum dado conforme ele desejar.

Público-alvo

Jovens, adultos, estudantes, bibliotecários, escolas, ou qualquer empresa que queria fazer uma gestão de livros que precise de um sistema para isso.

Principais funcionalidades

Na parte de configurar as rotas da API, temos o controller, ele vai configurar as rotas e passar para o service

- GET(ALL) para obter todas as informações
- GET(One) para obter apenas um dado específico
- POST para enviar os dados
- PUT para atualizar algum dado
- DELETE para apagar algum dado

Nesse projeto, o service está responsável tanto por validar as informações, quanto enviar para o banco de dados, por exemplo, quanto o usuário vai criar algo para mandar para o banco de dados, o service chama a DTO para validar o que o usuário escreveu, exemplo, se o usuário escreveu uma string, ou o usuário tentou enviar um dado vazio sem nada, o service faz verificações para ver se o usuário tentou acessar uma categoria que não existe, retornando “Categoria com id “`{id}`” não encontrada”.

Outra funcionalidade importante foi o schemas, antes de enviar para o banco de dados, o service chama ele, dentro do service ele é declarado no constructor, nesse código utilizou categoryModel para fazer essa conexão com o banco de dados, dentro do schemas ele irá ter algumas validações antes de enviar para o banco, por exemplo, lá é possível configurar se o dado é obrigatório ou não.

Outras funcionalidades muito importantes são os modules, responsáveis pela importação do controller, service, banco de dados e schemas.

Parte 2 – Tecnologias Utilizadas

Explique as tecnologias e bibliotecas encontradas no projeto.

Para cada uma, responda:

Qual sua finalidade?

Utilizamos o TypeScript para criar todo o back-end, junto com o nest para implementar algumas funcionalidade importante do projeto, que facilita a vida do programador, utilizou o Mongoose que facilitou a configuração do banco de dados, tornando mais prático e menos robusto, a DTO foi utilizada para validações dos dados do usuário, tirando a necessidade de escrever mais linhas para isso, foi utilizado o node para subir o servidor local para realizar testes, onde precisaria utilizar o PostMan para realizar esses testes. Além disso, temos o dist, onde ele irá converter o código feito em TypeScript para JavaScript antes de jogar o código em ambiente de produção. Temos tbm os packages Json, que faz as importações de todas as dependências do código. Temos o Docker que foi utilizado para subir as informações para o MongoDB. E por fim, temos o GitIgnore para ignorar algumas pastas pesadas como a nodeModules para não ser enviadas para o GIT, é possível baixar ela novamente através do NPM Install.

Onde ela é utilizada no projeto?

TypeScript foi utilizado em praticamente todo o código, sendo possível ver ele em arquivos terminados em .TS, é possível encontrar o nest nas importações, o Mongoose foi utilizado nos modules para fazer as importações e no schema, ele tbm está presente no service, pois chama o schema que por sua vez faz o uso do Mongoose para enviar para o Banco de dados. A DTO é possível encontrar na própria pasta chamada DTO, temos como por exemplo o create-category.dto e update-category.dto, o PostMan não está presente no projeto, mas foi utilizado para realizar testes, sendo muito importante destacalo, a dist podemos encontrar na própria pasta chamada dist. Os packages é possível encontrar pelo seu próprio nome e as terminações dos arquivos em. json. O Docker é possível encontrar pelo seu próprio nome em seu arquivo e sua extensão. yml. O Git ignore é possível encontrar através do próprio nome do arquivo.

Por que ela foi escolhida?

Em resumo, facilitar a vida do programador e utilizar tecnologia moderna e atual, poupa tempo e economiza linhas de código, ajuda na organização e distribuição de tarefas.

Parte 3 – Estrutura de Pastas e Arquivos

Analise toda a estrutura do projeto. Para cada pasta e arquivo, descreva:

Responsabilidade.

- Foi utilizado a arquitetura de camadas no código, onde cada camada tem sua responsabilidade.
- Module foi utilizado para importações de outros módulos, e tbm para importar o Mongoose.
- O controller foi utilizado para as configurações da rota da API, GET, POST, PUT, DELETE.
- Service foi utilizado para fazer validações e envio para o banco de dados.
- Schemas para configurar o banco de dados.
- DTO para validar os dados do usuário.
- DIST para converter TypeScript em JavaScript.
- Package para configuração das dependências .
- TsConfig para configuração, sendo utilizado o nodenext ao invés de comonJs.
- Docker para enviar para o MongoDB.
- GitIgnore para evitar enviar arquivos pesados para o GIT.

Função dentro do sistema.

Main – Cria o servidor e tudo o que a aplicação vai precisar.

Modules – Definição de importação, controller, service, banco de dados.

Controller – Definição de rota, função de GET ALL, GET ONE, POST, PUT, DELETE (Obter todos os dados, obter apenas um dado específico, criar um dado, atualizar um dado e apagar um dado).

Service – Todas as funções listadas no controller, mas com validações de dados e envio para o banco de dados.

Schemas – Configuração do banco de dados.

DTO – Validação dos dados.

Relação com outros componentes.

A main ela cria a aplicação.

Modules faz as importações e definição dos controller e service.

Controller configura as rotas da API e envia para o service.

O service faz a validação e usa o DTO para validar tbm, e envia para o banco de dados.

DTO – responsável por validar os dados.

Schemas faz a configuração do banco de dados.

Parte 4 – Fluxo de Funcionamento da API

Explique o fluxo de uma requisição desde a chegada da chamada HTTP até a resposta enviada ao cliente.

Apresente:

- 1. Entrada da requisição.**

Quando o cliente entra na aplicação através da porta 3000 nesse código, ele consegue visualizar todas as informações e dados vindo do banco de dados.

- 2. Controller responsável.**

O controller é acionado configurando as rotas da api, GET ALL, GET ONE, POST, PUT, DELETE, e passando as informações recebidas para o service.

- 3. Service acionado.**

O service é chamado recebendo os dados do controller, ele faz validações e envia para o banco de dados.

- 4. Acesso ao banco de dados.**

Banco de dados recebe as requisições através da porta, ele ler as requisições e envia de volta para a aplicação.

- 5. Retorno da resposta.**

A aplicação recebe a resposta e por sua vez envia para o usuário.

Parte 5 – Análise das Rotas

Documente as rotas existentes.

Para cada rota:

Método HTTP.

Comunicação através da porta 3000 do servidor, onde o cliente faz as requisições.

Endpoint.

Servidor recebe as requisições e envia de volta para o cliente.

• Objetivo.

Estabelecer comunicação entre o cliente e o servidor.

• Dados recebidos.

Recebimento de dados vindo do servidor.

• Dados retornados.

Conversão dos dados recebidos em elementos visuais para o cliente, deixando mais atrativo e bonito.

Parte 6 – Possíveis Erros e Problemas

Identifique possíveis falhas encontradas.

Explique:

- **O problema encontrado.**

Código muito redundante, poderia reutilizar funções já criadas, como por exemplo, o `NoutFoundException` que foi utilizado várias vezes no service.

Sobrecarregação de camada, o próprio Service apresenta duas funções, ele faz validação de dados e ainda envia para o banco.

Desorganização e poluição, algumas partes do código davam para ser melhor organizada como no próprio service.

- **O impacto.**

Código redundante gera linhas desnecessárias, que afeta no tempo da aplicação, além de atrapalhar na organização. Sobrecarregação de camada, faz com que o código fique mal organizado e poluído visualmente. Desorganização e poluição afeta na manutenção do próprio código.

- **Como corrigir.**

Reaproveitar as funções, criar uma camada específica para envio para o banco de dados, como o repository.

Parte 7 – Sugestões de Melhoria

Apresente possíveis melhorias e justifique cada proposta.

Reaproveitar as funções, criar uma camada específica para envio para o banco de dados, como o repository, deixando muito mais organizado e prático.